Best Free Models for Your Use Case (2026 Practical Choices)

Model	Size	Best For	Laptop Friendly
Microsoft Phi-2	2.7B	Logic + coding basics	✅ Very Good
Mistral AI Mistral 7B	7B	Strong overall coding	⚠️ Medium
Meta Llama 3 8B	8B	Great assistant	⚠️ Medium
Qwen Qwen2.5 Coder 7B	7B	Excellent coding	⚠️ Medium
Google Gemma 2B	2B	Lightweight	✅ Good

Recommended Laptop Minimum

Resource	Minimum
RAM	16GB
Storage	50GB free
GPU	Optional
OS	Windows / Ubuntu

STEP 1 Install Python

Install:

Python **Python 3.11**

Download:

Official website:

https://python.org

Important:

During install, check:

✅ Add Python to PATH

After install test:

```
python --version
```

Expected:

```
Python 3.11.x
```

STEP 2 Install Git

Install:

Git

Check:

```
git --version
```

STEP 3 Install VS Code

Install:

Visual Studio Code

Extensions later.

STEP 4 Install Ollama

Install:

Ollama -> install .exe file in ollama official website

Why:

- Run Phi-2 locally
- Test models easily
- Later use custom fine-tuned model

Install from official site.

After install:

```
ollama --version
```

STEP 5 Create Project Folder

```
mkdir coding-llm
cd coding-llm
```

STEP 6 Create Virtual Environment

```
python -m venv venv
venv\Scripts\activate
```

STEP 7 Install Libraries

Run:

```
pip install --upgrade pip
pip install transformers datasets peft accelerate torch sentencepiece
```

STEP 8 Test Phi-2 Quickly in Ollama

Run:

```
ollama run phi
```

If phi unavailable:

```
ollama pull phi
ollama run phi
```

Ask:

```
Write Python program for bubble sort
```

What to Expect on CPU

Task	Speed
Small prompt	OK
Code generation	Moderate
Fine-tuning	Slow
Inference	Usable

Goal of Your Coding Assistant

Since your users are **students**, dataset should teach:

- ✅ Write code
- ✅ Explain code
- ✅ Fix errors
- ✅ Debug programs
- ✅ Java / Python / C / Web
- ✅ Interview questions
- ✅ Best practices
- ✅ Step-by-step teaching style

Create file:

`coding_dataset.json`

Each line = one JSON object

Example:

```
{"instruction":"What is Python list?","input":"","output":"Python list is ordered mutable collection..."}
{"instruction":"Write Java factorial program","input":"","output":"public class Main {...}"}
{"instruction":"Explain React useState","input":"","output":"useState manages state in functional components..."}
```

NEXT STEP — Expand to 1000+ Records Automatically

Creating records manually is slow.

Smart engineers do this:

✅ Write some manually
✅ Generate many variations automatically
✅ Clean data
✅ Train faster

How to Create Fast (Best Method)

Daily create 50 records:

- 20 Python
- 10 Java
- 10 SQL
- 5 React
- 5 Interview

In 20 days = 1000 records

Method 1

Use Python script to create variations.

Example:

Original question:

```
Write Python program for factorial
```

Variations:

```
Create factorial program in Python
How to find factorial using Python?
Python factorial using loop
Factorial using recursion in Python
```

One question becomes many training samples.

STEP — Folder Structure

```
coding-llm/
 ├── coding_dataset.jsonl
 ├── generate_dataset.py
 └── expanded_dataset.jsonl
```

generate_dataset.py

```
import json

samples = [
  {
    "topic": "factorial",
    "questions": [
      "Write Python program for factorial",
      "Create factorial program in Python",
      "Find factorial using loop in Python",
      "Factorial using recursion in Python"
    ],
    "answer": """def factorial(n):
  if n == 0:
    return 1
  return n * factorial(n-1)

print(factorial(5))"""
  },
  {
    "topic": "palindrome",
    "questions": [
```

```
            "Write Python palindrome program",
            "Check palindrome in Python",
            "Python string palindrome example"
        ],
        "answer": """text = "madam"
print(text == text[::-1])"""
    }
]

with open("expanded_dataset.jsonl", "w", encoding="utf-8") as f:
    for item in samples:
        for q in item["questions"]:
            row = {
                "instruction": q,
                "input": "",
                "output": item["answer"]
            }
            f.write(json.dumps(row) + "\n")

print("Dataset Generated Successfully")
```

Run It

```
python generate_dataset.py
```

Output

Creates:

```
expanded_dataset.jsonl
```

Method 2 :

Use your real student FAQs:

Create categories:

Category	Samples
Python errors	200
Java programs	200
SQL	200
React	200
Interview	200

= 1000 records

Method 3

Use CSV from institute/student doubts.

Convert to JSONL.

GOLD DATASET FORMULA

70% Beginner Questions
20% Intermediate
10% Advanced

This makes model useful for students.

Your Task Now

Expand dataset to at least:

```
500 records
```

No need 1000 immediately.

Reply Once Finished:

```
500 DONE
```

STEP 6 — Fine-Tune Phi-2 on Your CPU Laptop

Because your laptop is:

```
Windows / 16GB RAM / CPU Only
```

We must use a **lightweight fine-tuning strategy**.

Important reality:

❌ Full Phi-2 training on CPU = too slow
❌ Normal LoRA training = painful on CPU
✅ Small subset training + PEFT possible
✅ Best path = prototype locally, bigger training later on GPU/cloud

So Step 6 is about creating a **working first fine-tune pipeline**.

Goal of this Step

Train Phi-2 with your coding dataset using:

✅ LoRA adapters
✅ Small batch size
✅ Low memory mode
✅ CPU-compatible settings

STEP A — Install Required Libraries

Inside virtual environment:

```
pip install torch transformers datasets peft accelerate trl
```

STEP B — Keep Dataset Ready

Example file:

`expanded_dataset.jsonl`

Each line:

```
{"instruction":"Write Python factorial program","input":"","output":"def factorial(n): ..."}
```

STEP C — Create Training Script

File:

`train_phi2.py`

```
from datasets import load_dataset

from transformers import AutoTokenizer, AutoModelForCausalLM

from transformers import TrainingArguments, Trainer

from transformers import DataCollatorForLanguageModeling

from peft import LoraConfig, get_peft_model


model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"


tokenizer = AutoTokenizer.from_pretrained(model_name)

tokenizer.pad_token = tokenizer.eos_token


model = AutoModelForCausalLM.from_pretrained(model_name)


lora_config = LoraConfig(
   r=8,
   lora_alpha=16,
   lora_dropout=0.05,
   bias="none",
   task_type="CAUSAL_LM"
)
```

```
model = get_peft_model(model, lora_config)

dataset = load_dataset("json", data_files="expanded_dataset.jsonl")["train"]

def tokenize(example):
    text = f"Instruction: {example['instruction']}\nAnswer: {example['output']}"
    return tokenizer(
        text,
        truncation=True,
        padding="max_length",
        max_length=128
    )

tokenized = dataset.map(tokenize)

training_args = TrainingArguments(
    output_dir="./tinyllama-coding-model",
    num_train_epochs=1,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=2,
    logging_steps=1,
    save_steps=20,
    learning_rate=2e-4,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)
```

)

trainer.train()

model.save_pretrained("./tinyllama-coding-model")

tokenizer.save_pretrained("./tinyllama-coding-model")

STEP D — Run Training

```
python train_phi2.py
```

What to Expect on CPU

Dataset Size	Approx Time
100 records	30–60 min
500 records	2–6 hrs
1000 records	6+ hrs

Depends on processor.

STEP — Chat with Your Own Coding Assistant Locally

Goal

Use your trained model like ChatGPT.

Ask:

✅ Write code
✅ Explain errors

✅ Teach concepts
✅ Solve student doubts

STEP A — Create Test Script

Create file:

`chat.py`

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_path = "./tinyllama-coding-model"

tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForCausalLM.from_pretrained(model_path)

while True:
    prompt = input("You: ")

    if prompt.lower() == "exit":
        break

    text = f"Instruction: {prompt}\nAnswer:"

    inputs = tokenizer(text, return_tensors="pt")

    outputs = model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        do_sample=True
    )
```

```
result = tokenizer.decode(outputs[0], skip_special_tokens=True)


print("\nAI:", result.replace(text, "").strip())

print()
```

STEP B — Run Chatbot

```
python chat.py
```

Ask Questions

Examples:

```
Write Python factorial program
Explain Java inheritance
What is SQL JOIN?
Fix Python IndexError
```

Exit:

```
exit
```

STEP 8A — Build React ChatGPT UI for Your Coding Assistant

Recommended Stack

Layer	Tech
Frontend	React
Styling	Tailwind CSS
API	FastAPI or Spring Boot
Model	Your TinyLlama

STEP 8A.1 Create React App

Run:

```
npx create-react-app coding-ai-ui
cd coding-ai-ui
npm install
```

STEP 8A.2 Replace `src/App.js`

Use this code:

```
import { useState } from "react";
import ReactMarkdown from "react-markdown";
import "./App.css";


function App() {
  const [message, setMessage] = useState("");
  const [chat, setChat] = useState([]);

  const sendMessage = async () => {
```

```
  if (!message.trim()) return;

  const newChat = [...chat, { sender: "user", text: message }];
  setChat(newChat);

  const res = await fetch("http://127.0.0.1:8000/ask", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ prompt: message })
  });

  const data = await res.json();

  setChat([
    ...newChat,
    { sender: "ai", text: data.response }
  ]);

  setMessage("");
};

  return (
```

```
    <div className="app">
      <h1>Karun AI Coding Mentor</h1>

      <div className="chat-box">
        {chat.map((msg, index) => (
          <div key={index} className={msg.sender}>
  {msg.sender === "ai" ? (
    <ReactMarkdown>{msg.text}</ReactMarkdown>
  ) : (
    msg.text
  )}
</div>
        ))}
      </div>

      <div className="input-box">
        <input
          value={message}
          onChange={(e) => setMessage(e.target.value)}
          placeholder="Ask coding question..."
        />
        <button onClick={sendMessage}>Send</button>
      </div>
    </div>
```

```
  );
}

export default App;
```

STEP 8A.3 Replace `src/App.css`

```
body {
  margin: 0;
  font-family: Arial;
  background: #0f172a;
  color: white;
}

.app {
  max-width: 900px;
  margin: auto;
  padding: 20px;
}

h1 {
  text-align: center;
}

.chat-box {
  height: 500px;
  overflow-y: auto;
```

```
  background: #1e293b;
  padding: 15px;
  border-radius: 10px;
}

.user {
  background: #2563eb;
  padding: 10px;
  margin: 10px 0;
  border-radius: 10px;
  text-align: right;
}

.ai {
  background: #334155;
  padding: 10px;
  margin: 10px 0;
  border-radius: 10px;
}

.input-box {
  display: flex;
  margin-top: 15px;
}

input {
```

```
  flex: 1;
  padding: 12px;
  font-size: 16px;
}

button {
  padding: 12px 20px;
  background: #22c55e;
  border: none;
  color: white;
  cursor: pointer;
}

code {
  background: #020617;
  padding: 4px 6px;
  border-radius: 5px;
  color: #22c55e;
}

pre {
  background: #020617;
  padding: 10px;
  border-radius: 8px;
  overflow-x: auto;
}
```

STEP 8B — Connect React UI to Your Real AI Model

We will build:

```
React Frontend  --->  FastAPI Backend  --->  TinyLlama Model
```

So instead of fake reply:

```
AI: hello
```

You get real coding answers.

Architecture

```
Browser Chat UI
      ↓
localhost:3000

React sends question
      ↓
localhost:8000/ask

Python loads your model
      ↓

Returns AI answer
```

STEP 8B.1 Install FastAPI

In your model project folder:

```
pip install fastapi uvicorn
```

STEP 8B.2 Create Backend File

Create:

```
api.py
```

from fastapi import FastAPI

from pydantic import BaseModel

from transformers import AutoTokenizer, AutoModelForCausalLM

```
from fastapi.middleware.cors import CORSMiddleware
import torch

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
model_path = "./tinyllama-coding-model"

print("Loading model...")

tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForCausalLM.from_pretrained(model_path)

class Question(BaseModel):
    prompt: str

@app.post("/ask")
def ask_ai(data: Question):
    text = f"""
You are a professional coding assistant.
Give clear and correct coding answers.

Instruction: {data.prompt}
```

```
Answer:
"""

    inputs = tokenizer(text, return_tensors="pt")

    with torch.no_grad():
        outputs = model.generate(
    **inputs,
    max_new_tokens=350,
    temperature=0.2,
    top_p=0.9,
    do_sample=True,
    repetition_penalty=1.15,
    no_repeat_ngram_size=3,
    pad_token_id=tokenizer.eos_token_id
)

    result = tokenizer.decode(outputs[0], skip_special_tokens=True)
    answer = result.replace(text, "").strip()

    return {"response": answer}
```

STEP 8B.3 Run Backend

```
uvicorn api:app --reload
```

Runs at:

```
http://127.0.0.1:8000
```